

Introduction à l'apprentissage automatique

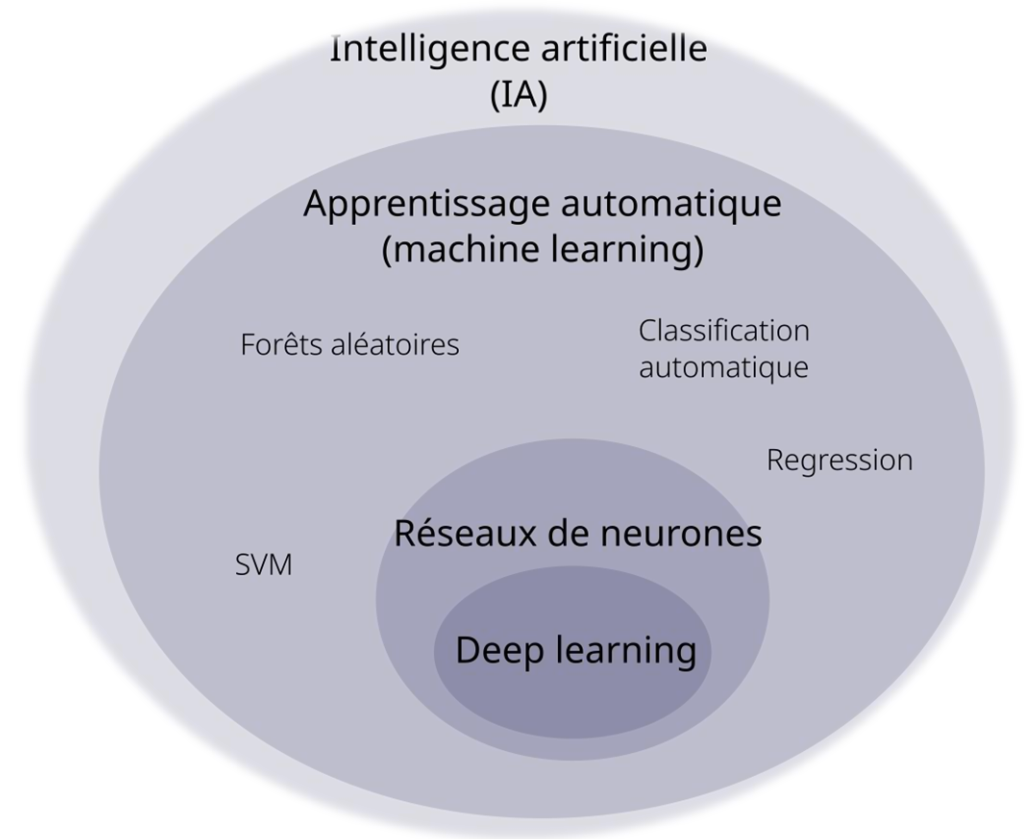
BUT Informatique — R4.A.L1 — 2024/2025

Alexis Juven

Introduction

Qu'est ce que l'apprentissage automatique ?

- Donner aux ordinateurs la capacité d'« apprendre » à partir de données, c'est-à-dire d'**améliorer leurs performances** à résoudre des tâches **sans être explicitement programmés** pour chacune.
- Programmation classique : logique prédéfinie qui résout une tâche « de manière prouvable ».
 - **Entrées + Programme → Sorties**
- *Machine learning* : modèle généré automatiquement à partir de données qui résout une tâche « en pratique ».
 - **Données (Entrées, Sorties) → Programme**



De nombreuses applications...

- Médecine : diagnostic automatique, imagerie
- Vision par ordinateur : reconnaissance d'image
- Voitures autonomes
- Algorithmes de recommandation
- Jeux compétitifs
- Traitement du langage naturel : Chat Bot, reconnaissance vocale, détection de spam
- Génération d'image, vidéo, son
- ...

... mais aussi de nombreux enjeux

- Biais discriminatoire dans les données d'entraînement
- Données personnelles, vie privée, droits d'auteur
- Concentration du pouvoir technologique, législation ?
- Suppression ou transformation d'emplois
- Génération de fausses informations/images/vidéos. Cyber attaques
- Armement autonome
- IA générale, *AI safety* et *value alignment* : comment s'assurer qu'un potentiel agent (super)intelligent agisse selon certaines valeurs morales et selon nos propres intérêts ?
- ...

Pour ce cours

Les outils à notre disposition

Langage Python : de nombreuses bibliothèques de traitement, d'analyse, et de visualisation des données.

- **Numpy, Pandas** : Manipulation, pré et post traitement des données
- **Matplotlib, Seaborn** : Visualisation des données, affichage des résultats
- **Scikit-learn** : boîte à outils pour l'apprentissage automatique et l'analyse de données. Implémente tous les modèles qui seront présentés.
- **Tensorflow (keras)**: bibliothèque pour l'apprentissage profond (réseaux de neurones)

Notations

- Variable en caractère fin ($x, y, \dots$) → **Nombres réels**.

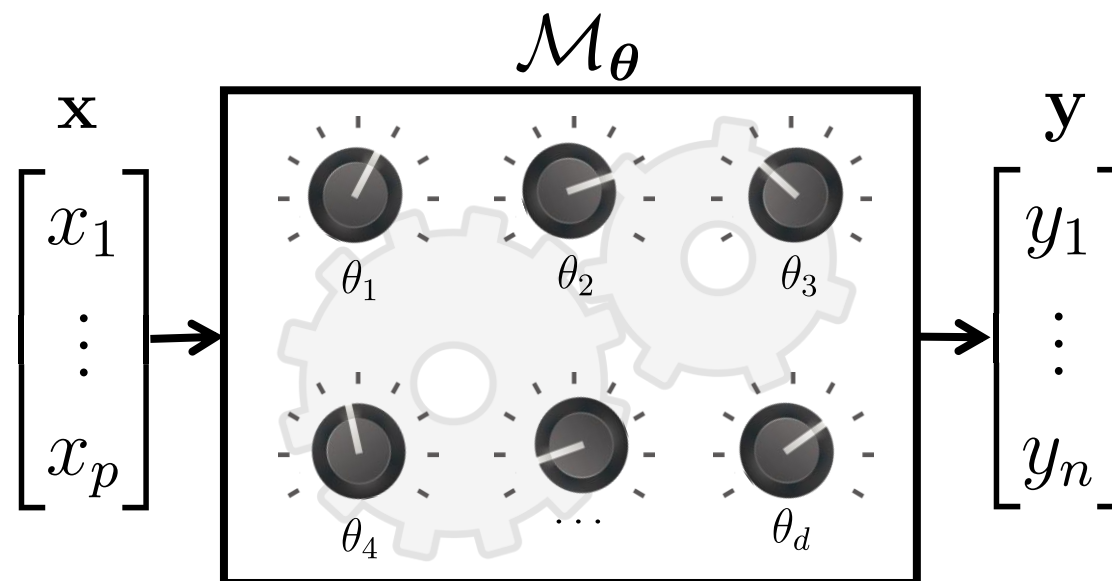
- Variable minuscule en gras → **Vecteurs**. (ex. $\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n$, vecteur de dimension n)

- Variable majuscule en gras → **Matrices**. (ex. $\mathbf{A} = \begin{bmatrix} a_{1,1} & \cdots & a_{1,p} \\ \vdots & \ddots & \vdots \\ a_{n,1} & \cdots & a_{n,p} \end{bmatrix} \in \mathbb{R}^{n \times p}$, matrice à n lignes et p colonnes)

Principe général et exemple simple

Principe général de l'apprentissage automatique

- On veut faire effectuer une tâche par un modèle $\mathcal{M}_\theta$ qui prend p entrées et renvoie n sorties.
- Le modèle est paramétré par d paramètres $\theta = (\theta_1, \dots, \theta_d)$ qui influent sur le calcul de la sortie $\mathbf{y}$ à partir de l'entrée $\mathbf{x}$.
- L'apprentissage, c'est utiliser des données pour trouver θ tel que $\mathcal{M}_\theta$ réalise la tâche souhaitée.



Ce qu'il nous faut

- Un jeu de données sur lequel entrainer le modèle, appelé jeu de données d'apprentissage.
- Un modèle $\mathcal{M}_\theta$ contenant des paramètres internes θ à apprendre.
- Une ou des métriques/erreurs pour quantifier à quel point le modèle réalise correctement ou non la tâche.
- Un algorithme d'apprentissage qui calcule ou modifie θ pour optimiser la métrique de performance de $\mathcal{M}_\theta$ sur les données d'apprentissage.

Exemple simple : la régression linéaire (dimension 1)

- On veut approximer $\{(x_1, y_1), \dots, (x_N, y_N)\}$ (nuage de points) par une fonction affine

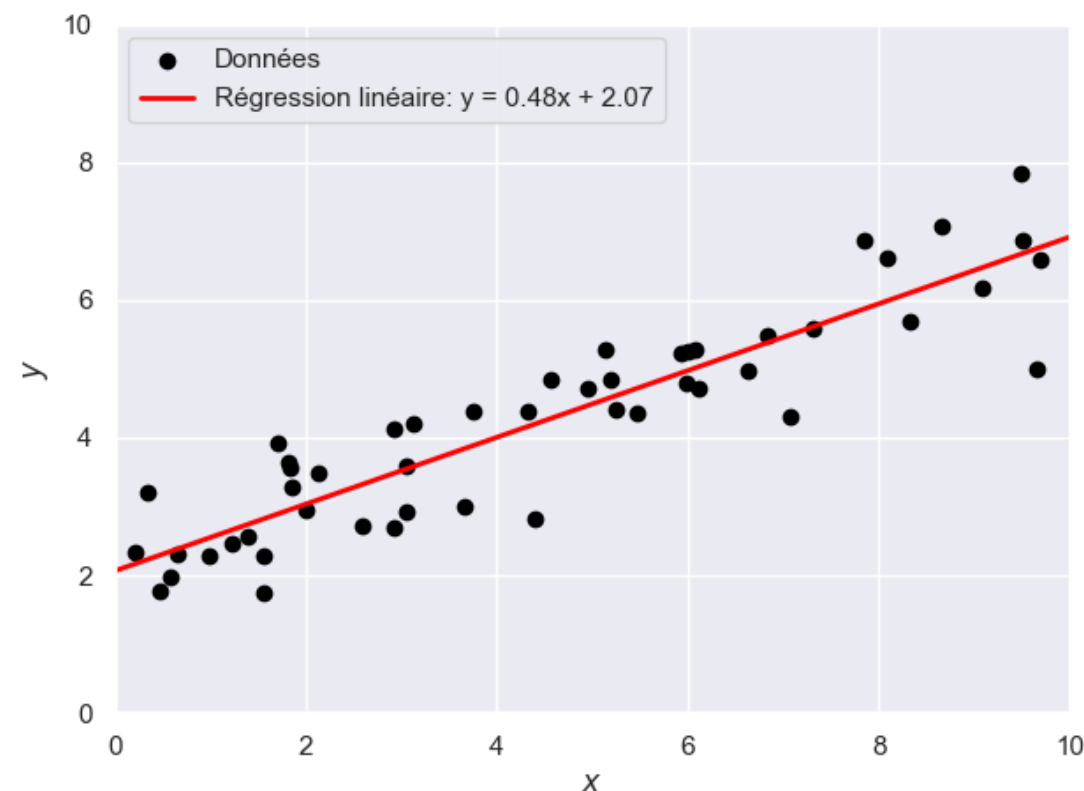
- Notre modèle est ici $\mathcal{M}_{a,b}(x) = ax + b$ et on cherche (a, b) tel que :

$$\forall k, \quad y_k \simeq ax_k + b$$

- Méthode des moindres carrés, on minimise l'erreur quadratique moyenne :

$$\text{MSE}(a, b) = \frac{1}{N} \sum_{k=1}^N (\mathcal{M}_{a,b}(x_k) - y_k)^2$$

- Optimiseur: calcul itératif, direct ?



Vaut 0 (donc est minimal) si le modèle renvoie exactement y_k

Régression linéaire, comment minimiser l'erreur ?

Premier algorithme d'apprentissage naïf :

- Choisir (a, b) aléatoirement
- A chaque étape, modifier aléatoirement a , et b d'un petit pas
- Si la modification réduit l'erreur quadratique moyenne (MSE), garder la modification
- Répéter

Très peu efficace, mais félicitations, on a fait de l'apprentissage automatique !

Pour faire mieux, on verra plus tard.

```
1  import numpy as np
2
3
4  def MSE(ys, ys_pred):
5      return np.mean((ys - ys_pred)**2)
6
7
8  def naive_linear_regression(xs, ys, nb_steps = 1000, eps = 0.1):
9
10     a = np.random.normal()
11     b = np.random.normal()
12     best_mse = MSE(ys, a * xs + b)
13
14     for _ in range(nb_steps):
15
16         new_a = a + np.random.normal(scale = eps)
17         new_b = b + np.random.normal(scale = eps)
18         new_mse = MSE(ys, new_a * xs + new_b)
19
20         if new_mse < best_mse:
21             a, b, best_mse = new_a, new_b, new_mse
22
23     return a, b
```

Vers des tâches plus complexes

- Pour réaliser des tâches plus complexes qu'une régression linéaire 1D, on pourra:
 - Complexifier le modèle $\mathcal{M}_\theta$
 - En lui faisant effectuer des calculs non-linéaires (+ de non-linéarités $\rightarrow$ + de complexité)
 - En augmentant drastiquement le nombre de paramètres dans θ (ex. ChatGPT : 175 milliards)
 - Trouver les bonnes métriques à minimiser/maximiser
 - Utiliser des algorithmes d'apprentissage plus intelligents (exemple : descente de gradient, calcul matriciel)
 - Augmenter la quantité de calculs (Parallélisme, GPUs, supercalculateurs, et long temps d'apprentissage)

Types d'apprentissage et exemples d'application

Apprentissage supervisé

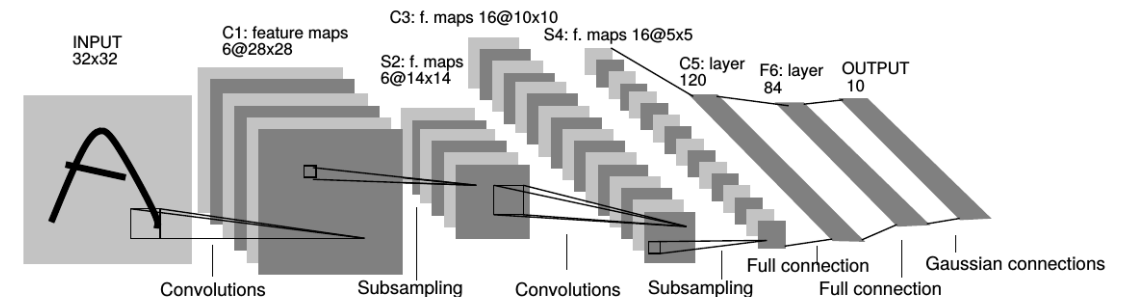
- Forme la plus répandue d'apprentissage. Un modèle apprend grâce à des couples « (entrée, sortie attendue) ». Nécessite souvent d'annoter manuellement un jeu de données.
- Deux types de tâche. Classification (catégorisation) et régression (approximation de fonction).
- Exemple : reconnaissance de caractères manuscrits sur des chèques

labels

données

« zéro » : 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
« un » : 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
« deux » : 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
... 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
6 6 6 6 6 6 6 6 6 6 6 6 6 6 6
7 7 7 7 7 7 7 7 7 7 7 7 7 7 7
8 8 8 8 8 8 8 8 8 8 8 8 8 8 8
9 9 9 9 9 9 9 9 9 9 9 9 9 9 9

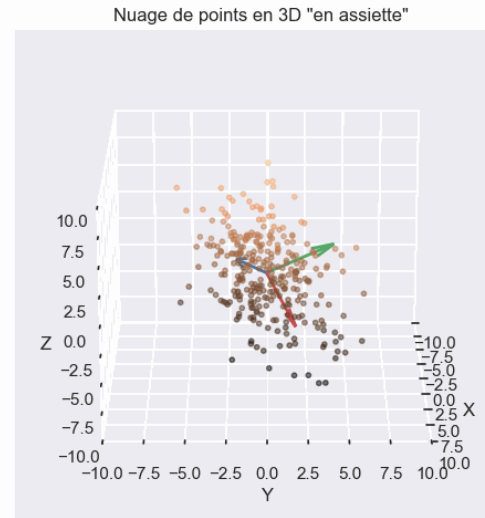
[MNIST dataset, Yann Le Cun, 1998]



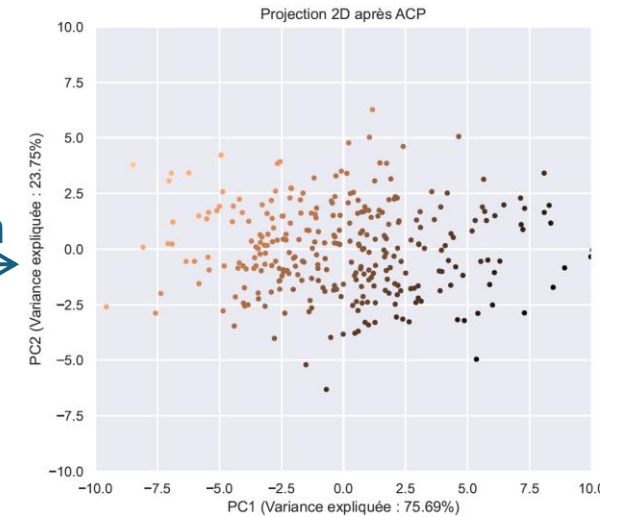
[LeNet, Yann Le Cun, 1998]

Apprentissage non supervisé

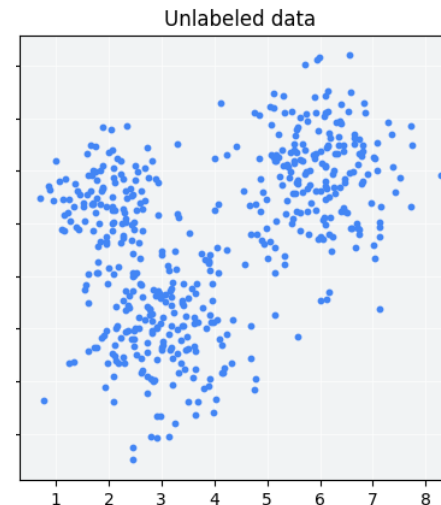
- Apprentissage à partir d'un jeu de données non annoté.
- Découverte automatique de structure sous-jacente (« motifs ») aux données.



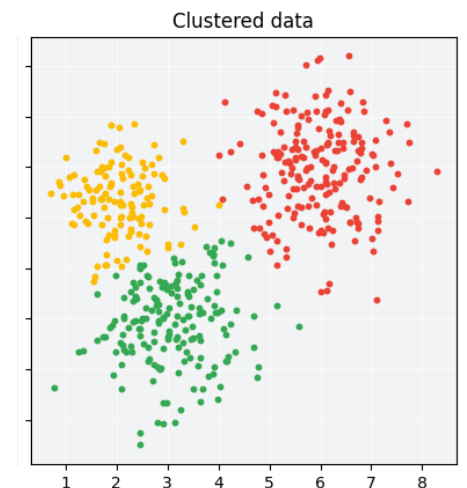
Réduction
de
dimension



- Exemple :
 - réduction de dimensionnalité des données
 - Clustering
 - Génération de données (ex. GPT)

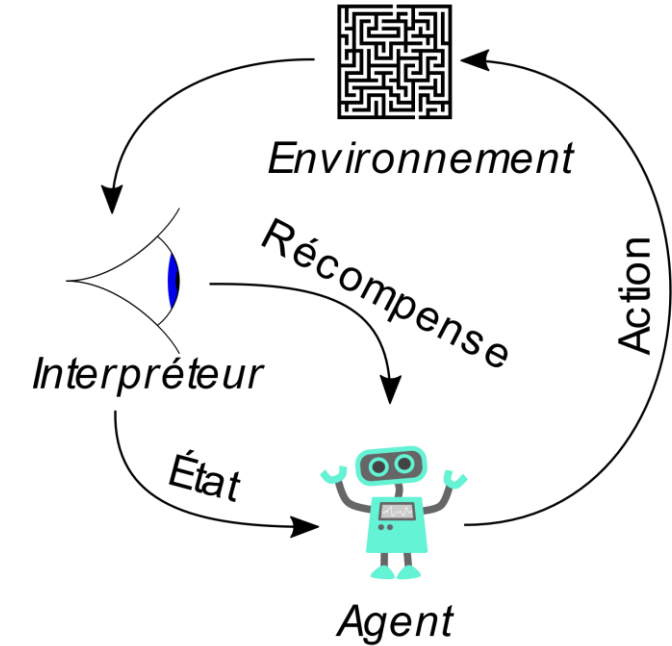


clustering

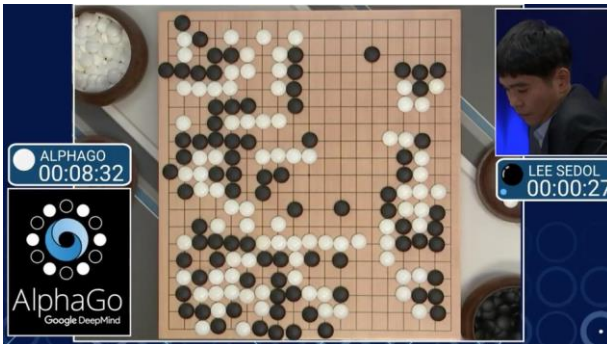


Apprentissage par renforcement

- Un agent agit au sein d'un environnement et apprend par essai-erreur à réaliser les actions qui maximisent sa récompense.
- Quelques exemples :

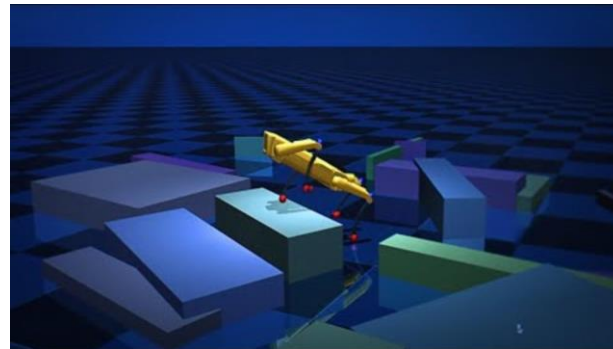


Jeu de GO



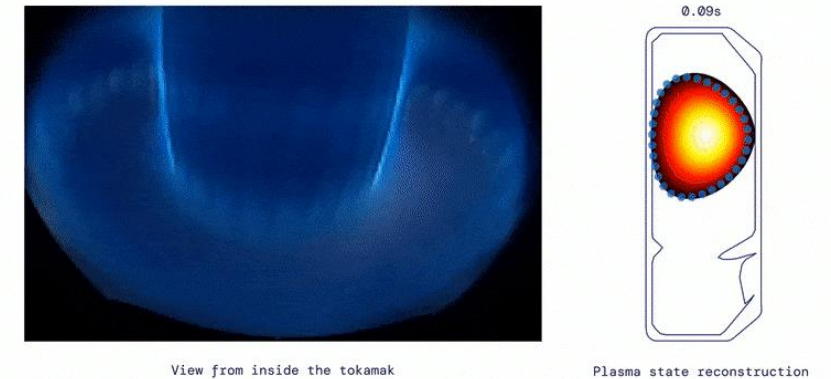
[DeepMind, 2016]

Robotique



[Boston Dynamics, 2024]

Contrôle commande en fusion nucléaire



[DeepMind, 2022]

Les trois grandes familles d'apprentissage

Apprentissage supervisé	Apprentissage non supervisé	Apprentissage par renforcement
On dispose de données annotées : des couples « (entrée, sortie attendue) ».	On dispose de données non annotées. On extrait automatiquement des motifs dans le jeu de donnée.	Un agent évolue et agit au sein d'un environnement. L'agent cherche à maximiser la récompense que lui donne l'environnement par essai-erreur.
Classification : donner une catégorie. Régression : approximer une fonction.	Réduction de la dimensionalité des données. Explication des données, détection d'anomalie. Génération de données.	Maximisation d'un objectif.
- Regression linéaire/logistique - SVM - Arbre de decision, forêt aléatoire - K-NN - MLP, CNN, LSTM, Transformer	- PCA - K-means - GMM - Autoencodeurs - GANs	- Q-learning - TD learning - Deep Q-Networks

Concepts fondamentaux

Processus usuel d'apprentissage

1. Création / chargement du jeu de données
2. Choix d'un modèle à utiliser et de l'algorithme d'apprentissage.
3. Prétraitement des données : extraction de *features*, encodage, normalisation, gestion des valeurs manquantes ou aberrantes
4. Séparation en données d'entraînement et de test
5. Entraînement du modèle sur les données d'entraînement (ex. algorithme itératif qui minimise une fonction de perte/erreur)
6. Evaluation des performances du modèle entraîné sur l'ensemble de test selon certaines métriques.
7. Déploiement du modèle

Normalisation des données

Certains modèles nécessitent des données dans des plages de valeurs cohérentes, surtout pour des données à plusieurs dimensions et d'unités différentes. Pour cela, on transforme les données $\mathbf{x}_1, \dots, \mathbf{x}_N$ en données normalisées $\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_N$

- Standardisation : valeurs de moyenne 0 et d'écart type 1

$$\tilde{\mathbf{x}}_k \leftarrow \frac{\mathbf{x}_k - \boldsymbol{\mu}_X}{\boldsymbol{\sigma}_X}$$

$$\boldsymbol{\mu}_X = \frac{1}{N} \sum_{k=1}^N \mathbf{x}_k$$

(moyenne)

$$\boldsymbol{\sigma}_X = \sqrt{\frac{1}{N} \sum_{k=1}^N (\mathbf{x}_k - \boldsymbol{\mu}_X)^2}$$

(écart type)

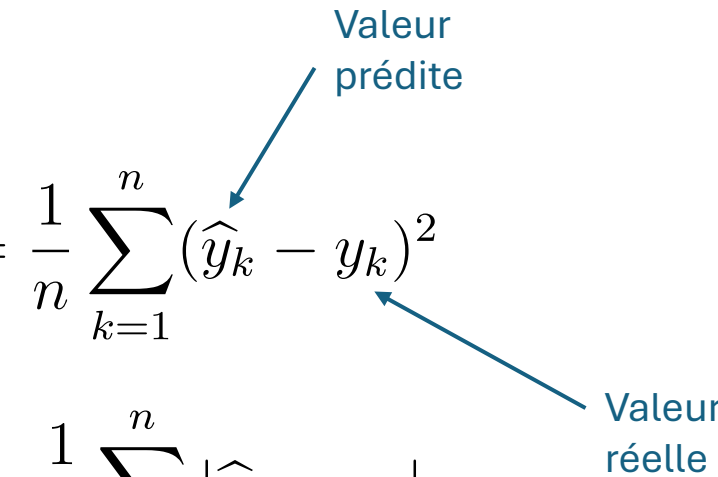
- *Min-Max scaling* : valeurs comprises entre 0 et 1

$$\tilde{\mathbf{x}}_k \leftarrow \frac{\mathbf{x}_k - \min_X}{\max_X - \min_X}$$

Fonction de perte/coût/d'erreur

- Fonction que cherche à minimiser le modèle lors de l'apprentissage supervisé.
Loss function en anglais.

- Quelques fonctions de perte

- MSE (*Mean Squared Error*) en régression :
$$\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum_{k=1}^n (\hat{y}_k - y_k)^2$$


The diagram shows the MSE formula with two blue arrows. One arrow points from the text 'Valeur prédite' to the predicted value $\hat{y}_k$ in the formula. The other arrow points from the text 'Valeur réelle' to the real value y_k in the formula.
- MAE (*Mean Absolute Error*) en régression :
$$\mathcal{L}_{\text{MAE}} = \frac{1}{n} \sum_{k=1}^n |\hat{y}_k - y_k|$$
- *Cross-Entropy* en classification :
$$\mathcal{L}_{\text{Cross-Entropy}} = - \sum_{k=1}^n y_k \log(\hat{y}_k)$$

Métriques de classification (binaire → positif ou négatif)

- **Précision**: proportion des valeurs prédites comme **P** qui sont bien **P**, utilisé si **FP coûteux** (ex. détection d'e-mail de spam)

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Rappel** (recall) : proportion des valeurs réellement **P** qui sont prédites comme **P**, utilisé si **FN coûteux** (ex. diagnostic médical)

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **Exactitude** (accuracy): proportion des valeurs correctement catégorisées, peu utile si classes déséquilibrées

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **F1 score** : utile si classes déséquilibrées

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

		Actual Value	
		Positive	Negative
Predicated Value	Positive	TP	FP
	Negative	FN	TN

Confusion matrix

Hyperparamètres

- Hyperparamètres : paramètres **non appris** par le modèle mais définis avant l'entraînement.
- Exemple : architecture du modèle, nombre d'étapes d'entraînement, valeurs limites, pas d'apprentissage...
- Contrairement aux paramètres du modèle, les hyperparamètres doivent être réglés manuellement, ou déterminés par des méthodes d'optimisation (recherche par grille, aléatoire, optimisation bayésienne, algorithmes génétiques).

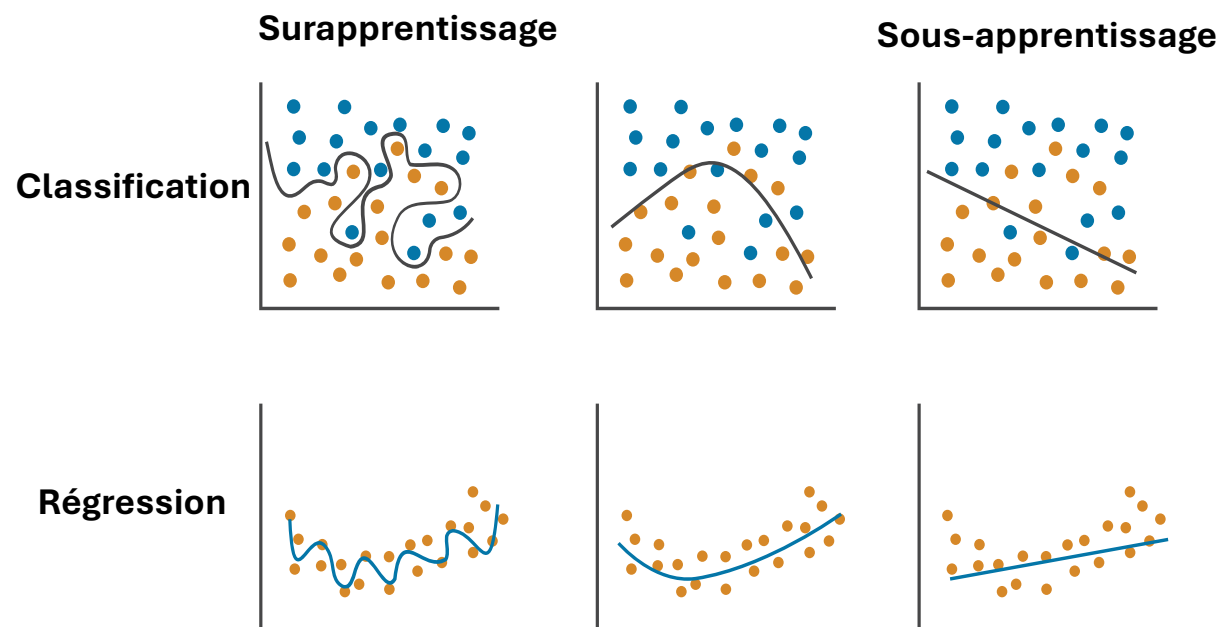
Surapprentissage et sous-apprentissage

- **Surapprentissage** (*overfitting*) : le modèle mémorise les données d'entraînement, dont le bruit, au lieu d'extraire uniquement des patterns généralisables.
 - Modèle trop complexe par rapport aux données ou trop d'itérations d'entraînement

Performance excellente sur l'entraînement
mais mauvaise sur les nouvelles données

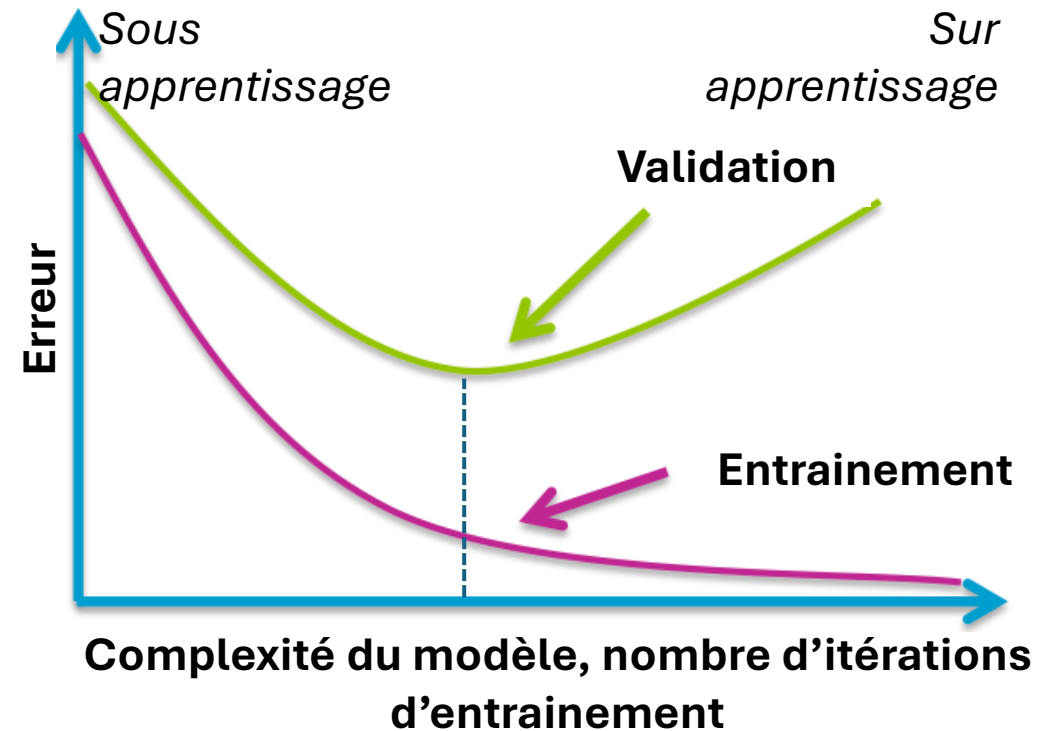
- **Sous-apprentissage** (*underfitting*) : modèle trop simple pour capturer les patterns des données.
 - Pas assez d'itération d'entraînement ou modèle trop linéaire pour des données complexes

Erreur élevée sur les ensembles
d'entraînement et de test



Ensembles d'entraînement, de validation, de test

- Lors de l'apprentissage d'un modèle, il est crucial de tester sa capacité à généraliser à de nouvelles données. On divise donc le jeu de données en trois ensembles :
 - **Ensemble d'entraînement** : 60 à 80 % des données, utilisé pour ajuster les paramètres du modèle.
 - **Ensemble de validation** : 10 à 20 % des données, utilisé pour choisir les hyperparamètres du modèle, tester la capacité de généralisation du modèle pendant l'entraînement, et potentiellement arrêter l'entraînement.
 - **Ensemble de test** : 10 à 20 % des données, utilisé pour évaluer les performances finales du modèle sur des nouvelles données.

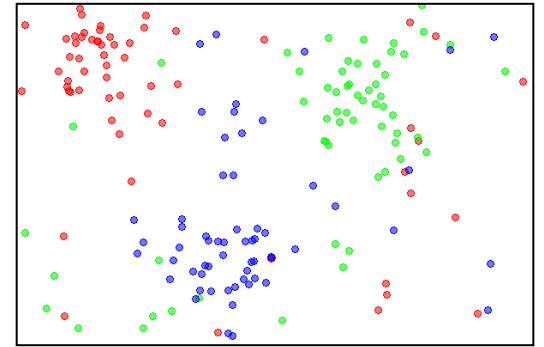


Quelques modèles classiques

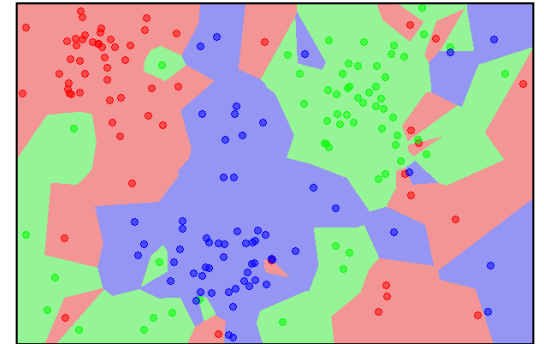
K Nearest Neighbors (k-NN)

- Modèle de classification en apprentissage supervisé : renvoie la classe majoritaire dans les k plus proches voisins d'un point.
- La frontière de décision entre deux classes change en fonction de k . Risque de surapprentissage si k trop faible et de sous-apprentissage si k trop grand.
- Lourd à calculer si trop de données d'entraînement. Peu efficace en grande dimension (malédiction de la dimension). Peu nécessiter une réduction de la dimensionalité.
- Peut aussi être utilisé en régression en interpolant les valeurs de k plus proches voisins.

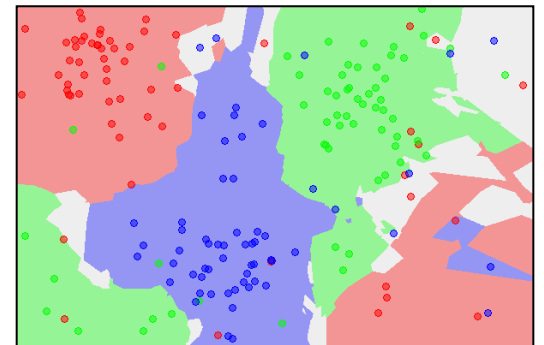
**Jeu
d'entraînement**



**Classification
1-NN**

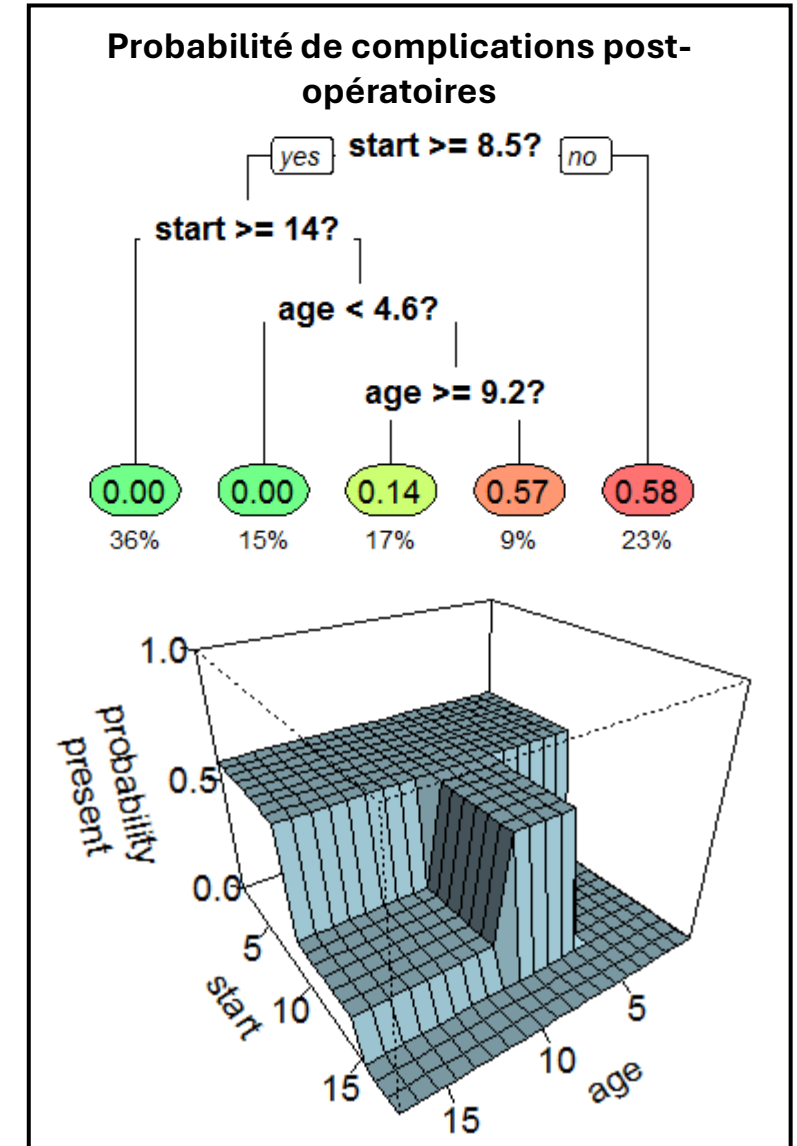


**Classification
5-NN**

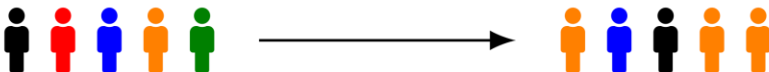


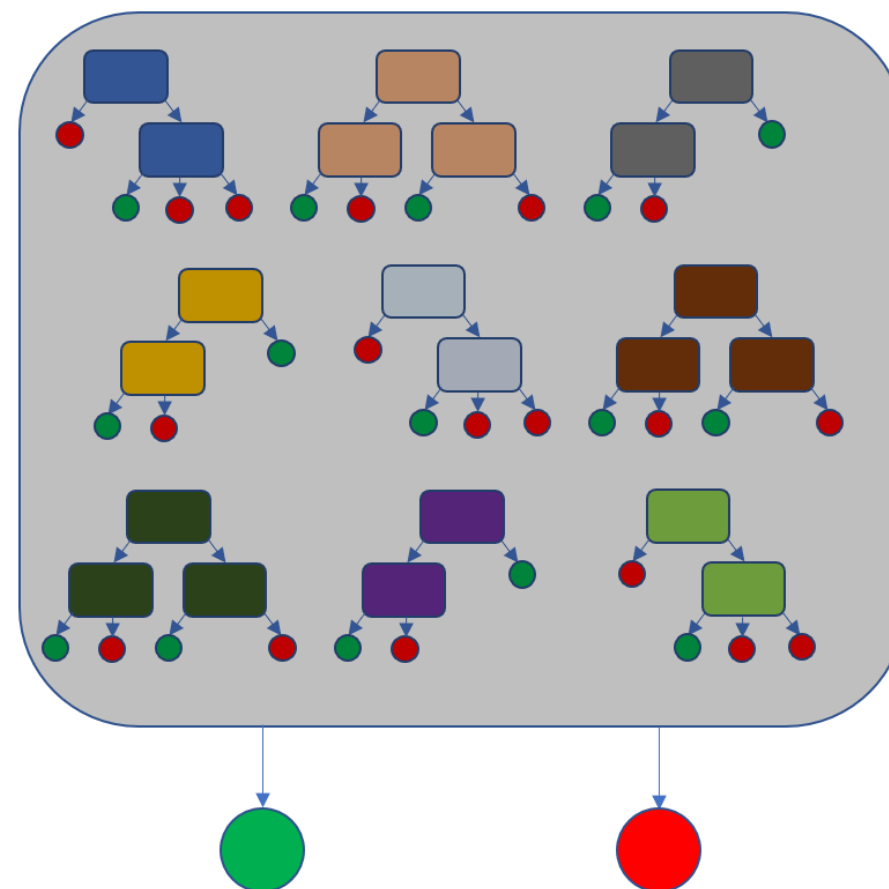
Arbre de décision

- Sélection d'une variable à chaque nœud de l'arbre et comparaison à un seuil. Dans chaque feuille renvoie la proportion des données pour chaque classe.
- Avantage : explicabilité de la prédiction (« white box »), peu de ressources de calcul. Limitation : modèle simple, arbre potentiellement gros pour gérer des données complexes, *overfitting*.
- Algorithme d'apprentissage : « approche top-down », démarre à chaque étape une feuille de l'arbre en un nœud. Choix de la feuille, de la variable et du seuil qui minimise ou maximise un certain critère. (Gini impurity, Information gain, variance)



Forêt aléatoire

- Ensemble d'arbres de décision
- Chaque arbre est entraîné sur des données légèrement différentes:
 - En effectuant un tirage aléatoire des données d'entraînement avec remise (*bootstrap aggregating*)

 - En sélectionnant aléatoirement qu'un sous ensemble de *features* (ex. *age*, *genre*, *poids...*) à utiliser pour faire sa prédiction (*feature bagging*)
- La prédiction de la forêt est faite en utilisant la prédiction de chaque arbre (« vote »).
- Efficace et robuste à l'*overfitting*, perd l'explicabilité



Random Forest

Classification naïve bayésienne

- Classification multiclasse $c_1, \dots, c_n$ d'une entrée $\mathbf{x}$ en utilisant la formule de Bayes des probabilités conditionnelles:

Probabilité pour une entrée $\mathbf{x}$ d'être dans la classe c_k $\longrightarrow$

$$p(c_k|\mathbf{x}) = \frac{p(\mathbf{x}|c_k)p(c_k)}{p(\mathbf{x})}$$

Proportion de données de type $\mathbf{x}$ au sein de la classe c_k dans le jeu d'entraînement $\longleftarrow$ $p(\mathbf{x}|c_k)$

Proportion de données au sein de la classe c_k dans le jeu d'entraînement $\longleftarrow$ $p(c_k)$

Proportion de données de type $\mathbf{x}$ au sein du jeu d'entraînement $\longleftarrow$ $p(\mathbf{x})$

- Entraînement fait en comptant les différentes proportions dans le jeu d'entraînement.
- Exemple : détection d'e-mail de spam

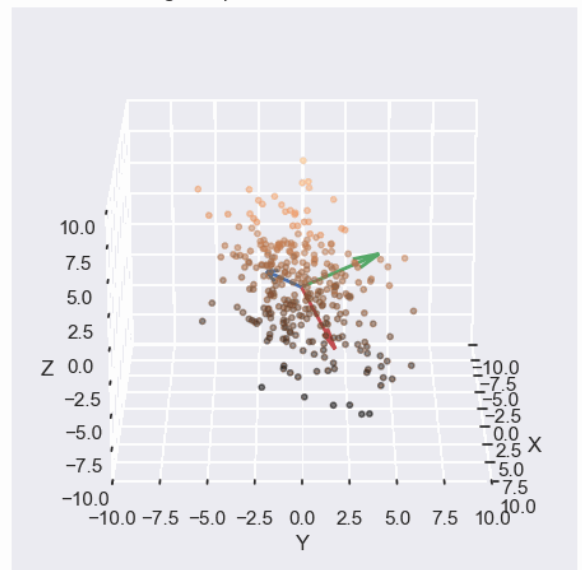
$$p(\text{Spam}|\text{mot}_1, \dots, \text{mot}_m) = \frac{p(\text{mot}_1|\text{Spam}) \cdots p(\text{mot}_m|\text{Spam})p(\text{Spam})}{p(\text{mot}_1) \cdots p(\text{mot}_m)}$$

- Suppose que les mots apparaissent indépendamment des autres

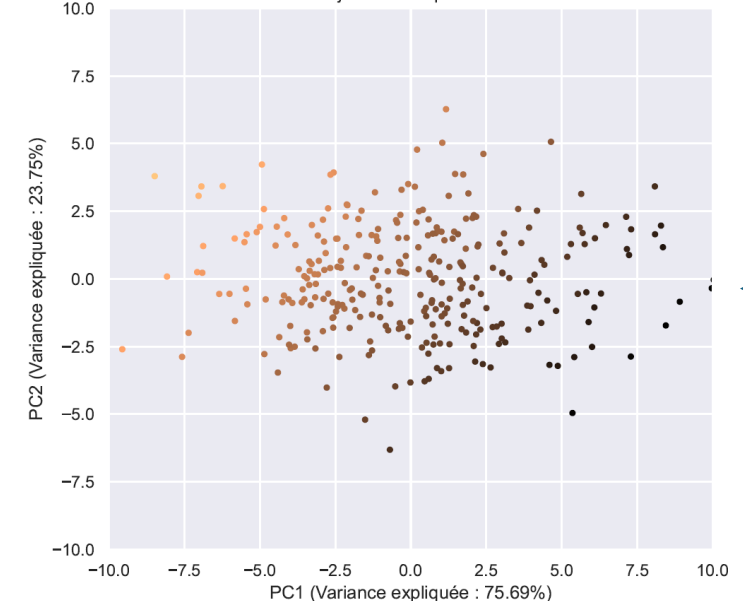
Analyse en Composante Principale (ACP)

- Apprentissage **non-supervisé** pour **réduire la dimension** des éléments d'un nuage de points. Crucial pour pouvoir utiliser des méthodes nécessitant une faible dimension.
- Idée : trouver un **nouveau système de coordonnées** perpendiculaires dans laquelle l'étalement des données (variance) est maximal selon la première coordonnée, puis selon la deuxième, etc. En ne **gardant que les premières coordonnées**, on réduit la dimension des données tout en préservant le maximum d'information.
- La transformation est **linéaire**, c'est-à-dire donnée par une matrice.

Nuage de points en 3D "en assiette"



Projection 2D après ACP



La régression linéaire (cas général)

- Dans le cas où on cherche à approximer une fonction $\mathbf{f} : \mathbb{R}^p \rightarrow \mathbb{R}^n$ à p entrées et n sorties à partir d'un jeu d'entraînement $\{(\mathbf{x}_1, \mathbf{y}_1), \dots, (\mathbf{x}_N, \mathbf{y}_N)\}$ avec $\mathbf{x}_k \in \mathbb{R}^p$ et $\mathbf{y}_k \in \mathbb{R}^n$

- On cherche une matrice $\mathbf{A} \in \mathbb{R}^{n \times p}$ (n lignes, p colonnes) et un vecteur $\mathbf{b} \in \mathbb{R}^n$ tels que :

$$\forall k, \quad \mathbf{y}_k \simeq \mathbf{A}\mathbf{x}_k + \mathbf{b}$$

- Méthode des moindres carrés ordinaires (**minimise la MSE**), calcul direct matriciel :

- En posant $\mathbf{X} = \begin{bmatrix} \mathbf{x}_1 & \cdots & \mathbf{x}_N \\ 1 & \cdots & 1 \end{bmatrix} \in \mathbb{R}^{(p+1) \times N}$ et $\mathbf{Y} = [\mathbf{y}_1 \quad \cdots \quad \mathbf{y}_N] \in \mathbb{R}^{n \times N}$

- Alors :
$$[\mathbf{A} \quad \mathbf{b}] = \mathbf{Y} \underbrace{\mathbf{X}^\top (\mathbf{X} \mathbf{X}^\top)^{-1}}_{\text{Matrice pseudo-inverse de } \mathbf{X}}$$

Matrice pseudo-inverse de $\mathbf{X}$

La régression polynomiale (dimension 1)

- On veut approximer $\{(x_1, y_1), \dots, (x_N, y_N)\}$ (nuage de points) par un polynôme de degré d

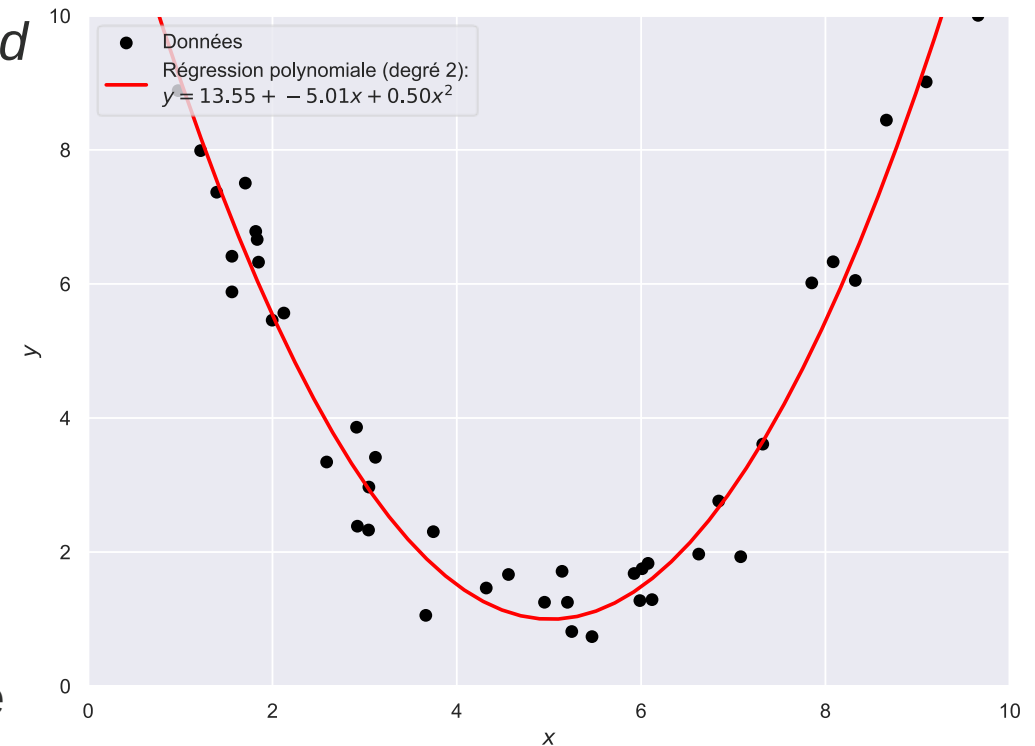
- On cherche $(a_0, \dots, a_d)$ tel que

$$\forall k, \quad a_0 + a_1 x_k + \dots + a_d x_k^d \simeq y_k$$

- En méthode des moindres carrés :

- On calcule $\forall k, \quad \mathbf{x}_k \leftarrow (1, x_k, x_k^2, \dots, x_k^d)$
- Alors les coefficients (a_i) sont les solutions de la régression **linéaire** par moindre carré sur

$$\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$$

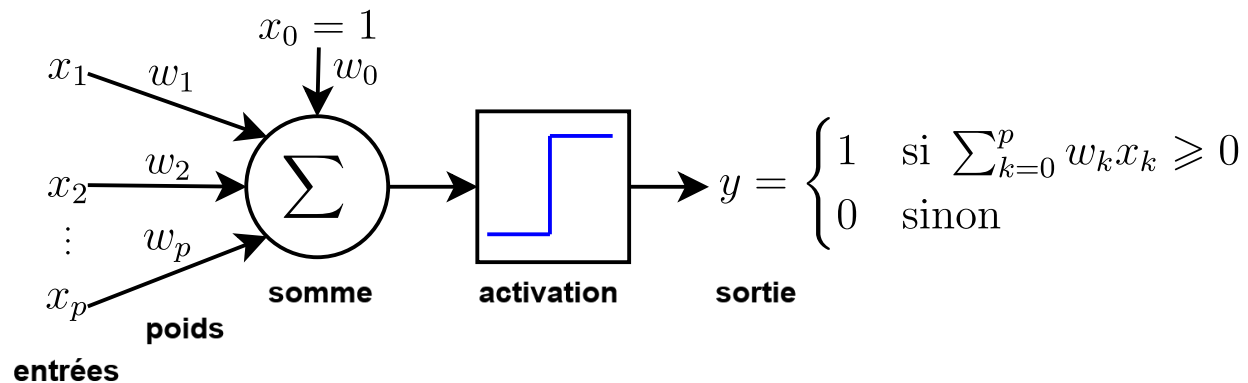


Perceptron et SVM, classification linéaire

Le perceptron : analogie d'un neurone

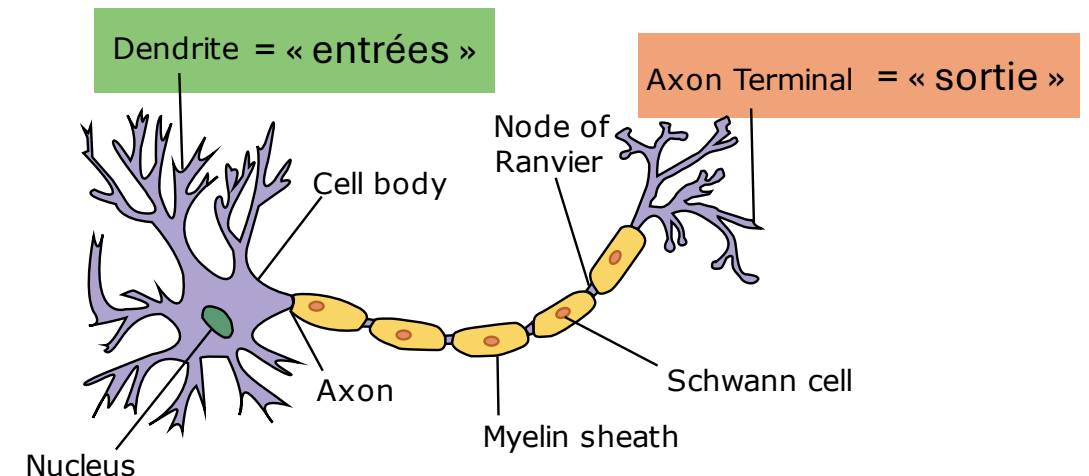
Perceptron (1957)

- Reçoit p entrées $x_1, \dots, x_p$
- Somme ces entrées en les pondérant par des poids $w_1, \dots, w_p$ et en ajoutant un biais w_0
- Transmet cette somme à une fonction d'activation (fonction de Heaviside) : renvoie 1 si la somme est positive, 0 sinon.



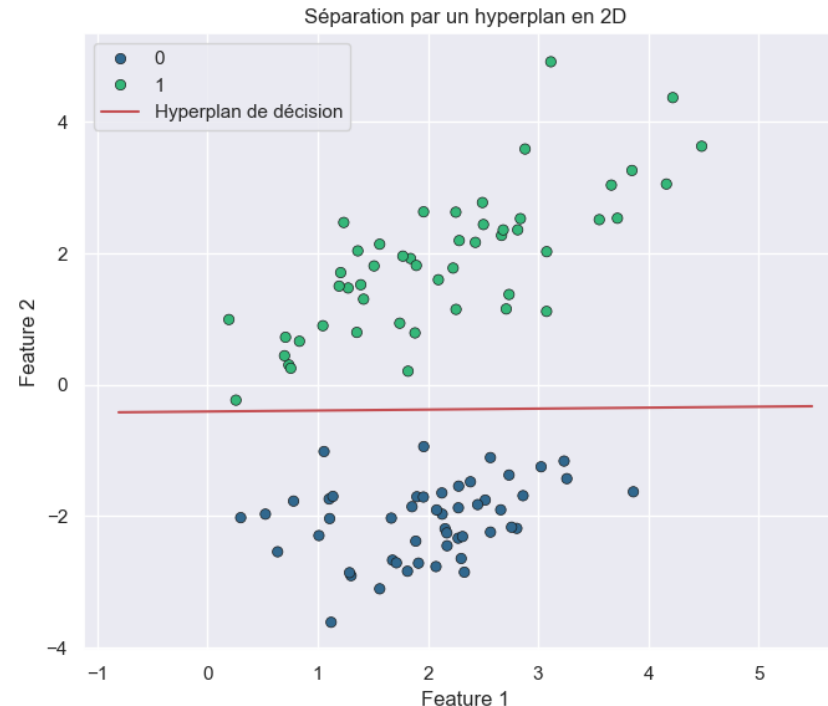
Neurone biologique

- Les dendrites captent les signaux électriques des autres neurones connectés
- Le corps cellulaire intègre les signaux
- Si il y a assez de stimulation, un signal de sortie est déclenché et transmis aux autres neurones via l'axone

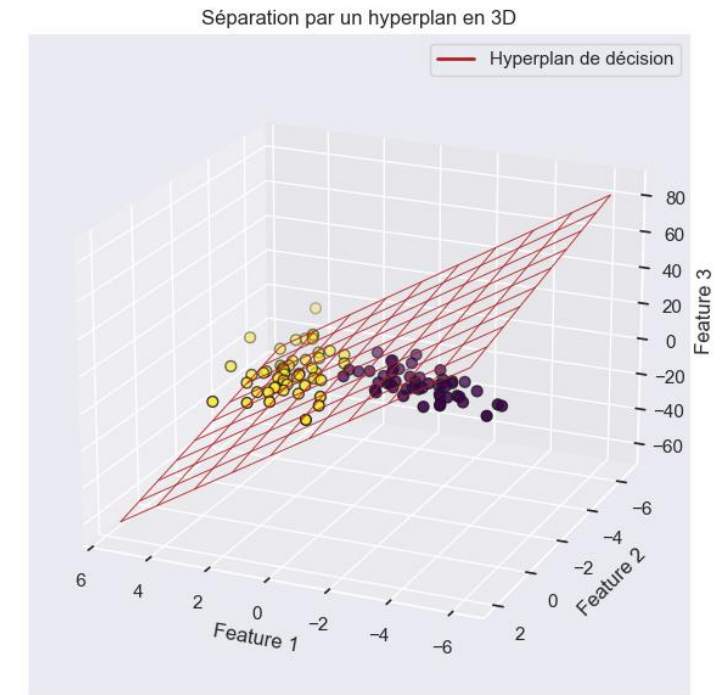


Le perceptron : un hyperplan séparateur

- Un perceptron ne fait que séparer l'espace en 2 par un hyperplan affine (généralisation d'un plan en dimension quelconque).
- Peut résoudre les problèmes de classification dits « linéairement séparables ».



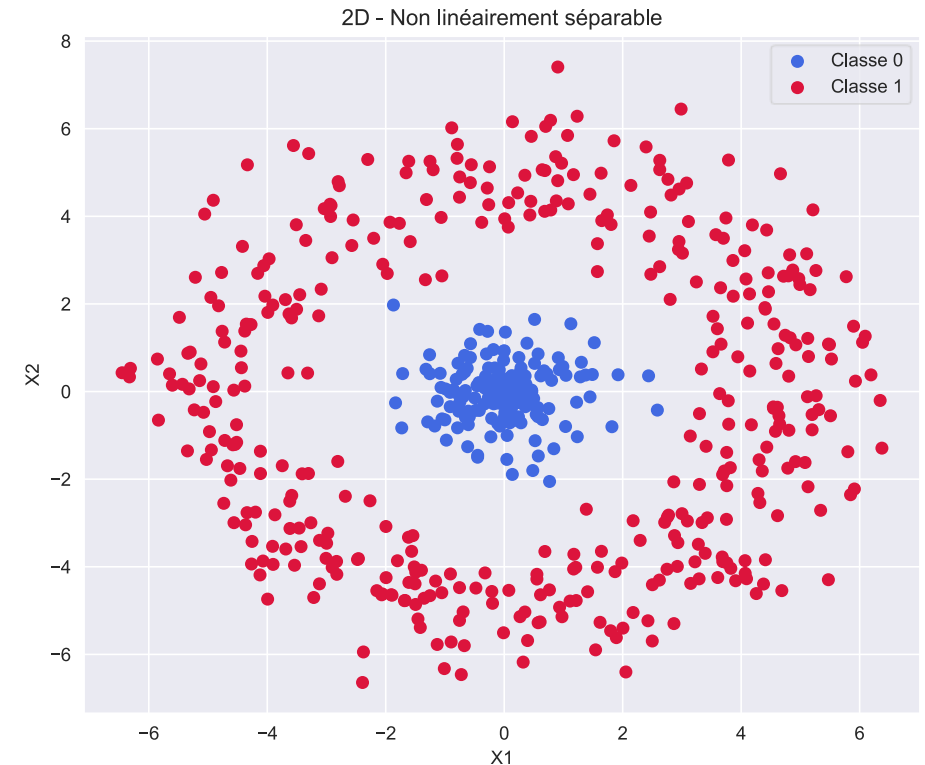
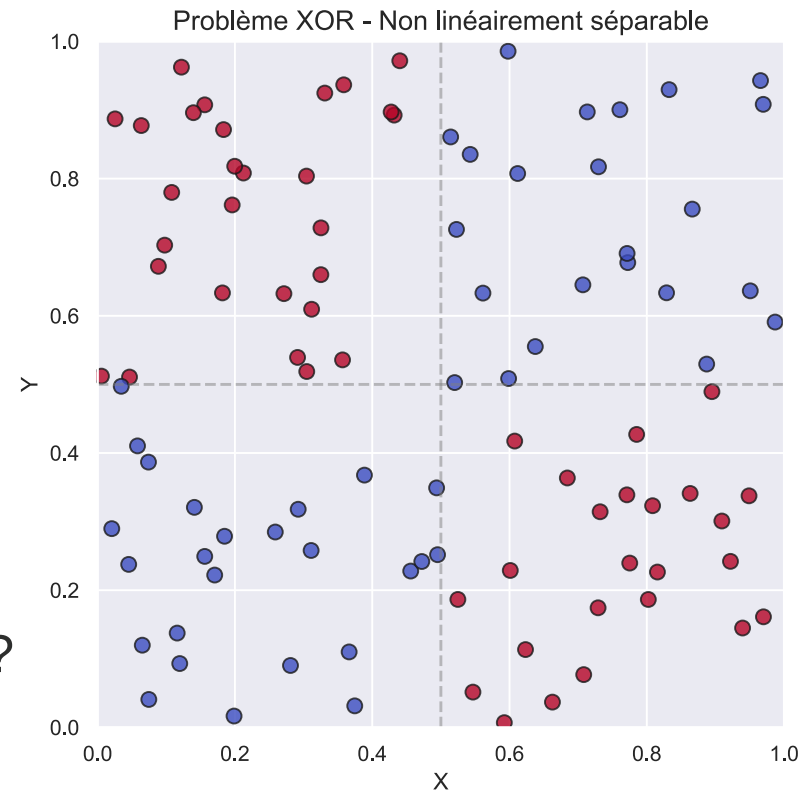
Perceptron à deux entrées



Perceptron à trois entrées

Problème non linéairement séparable

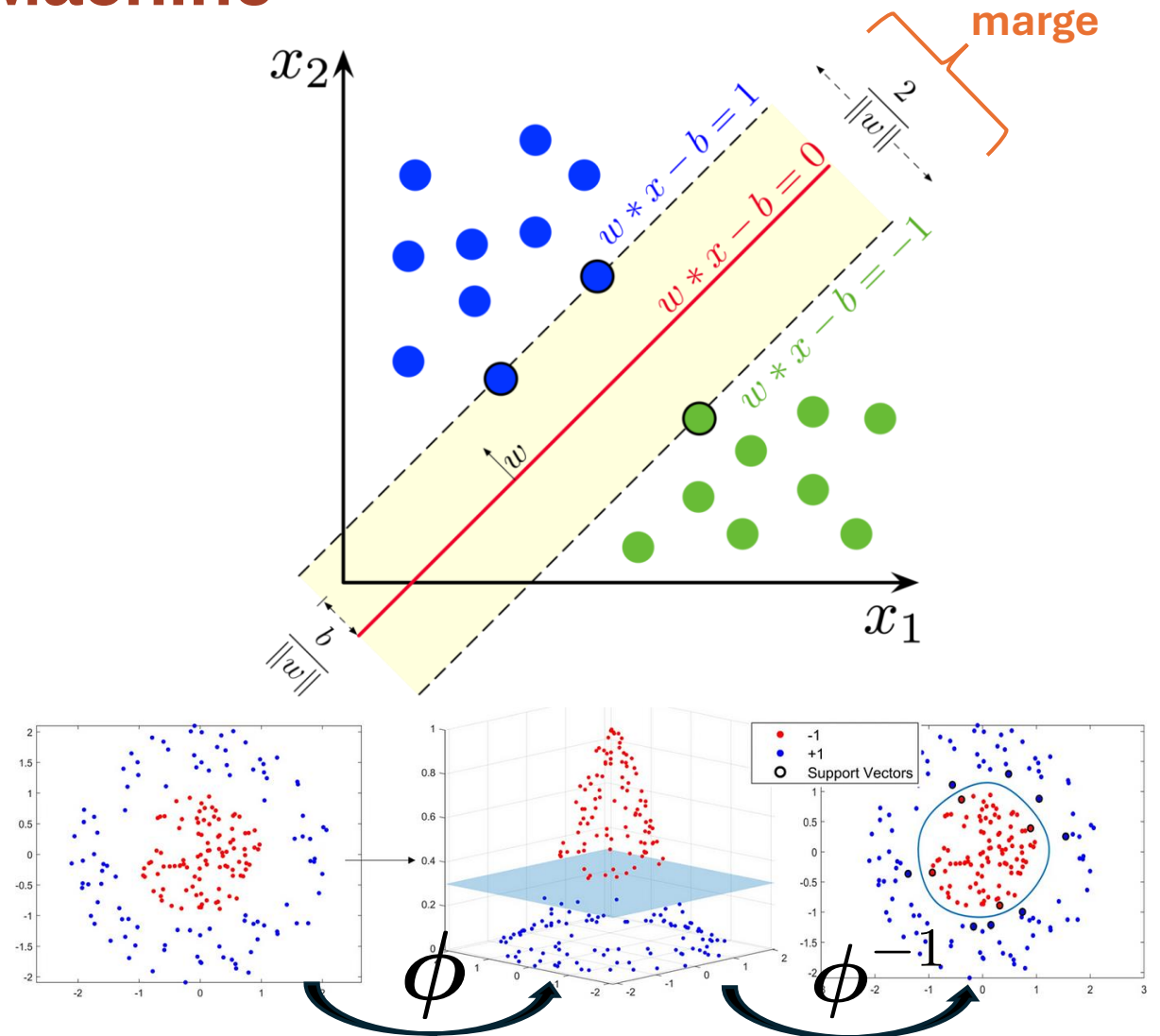
- Exemple de problème non linéairement séparable.
- Non solvable par un perceptron seul.
- Quel modèle utiliser ?



Le SVM : Support Vector Machine

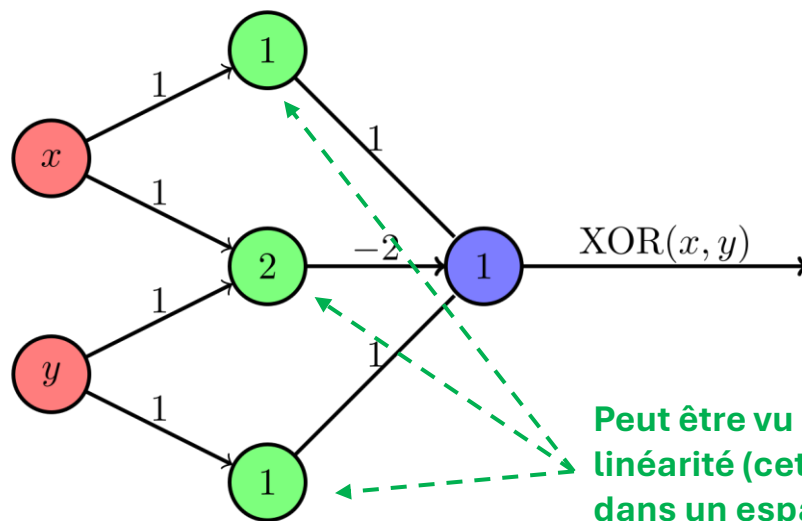
- Modèle similaire au perceptron (hyperplan de séparation de l'espace, renvoie $\{-1,1\}$ au lieu de $\{0,1\}$).
- L'algorithme d'apprentissage en classification maximise la **marge** entre les deux classes
- Grâce à une transformation non linéaire ϕ de ses entrées dans un espace de plus grande dimension, peut résoudre des problèmes **non linéairement séparables**.
- Astuce du noyau (*kernel trick*): pas besoin de calculer ϕ , il suffit d'avoir une fonction *noyau* K telle que :

$$K(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$$

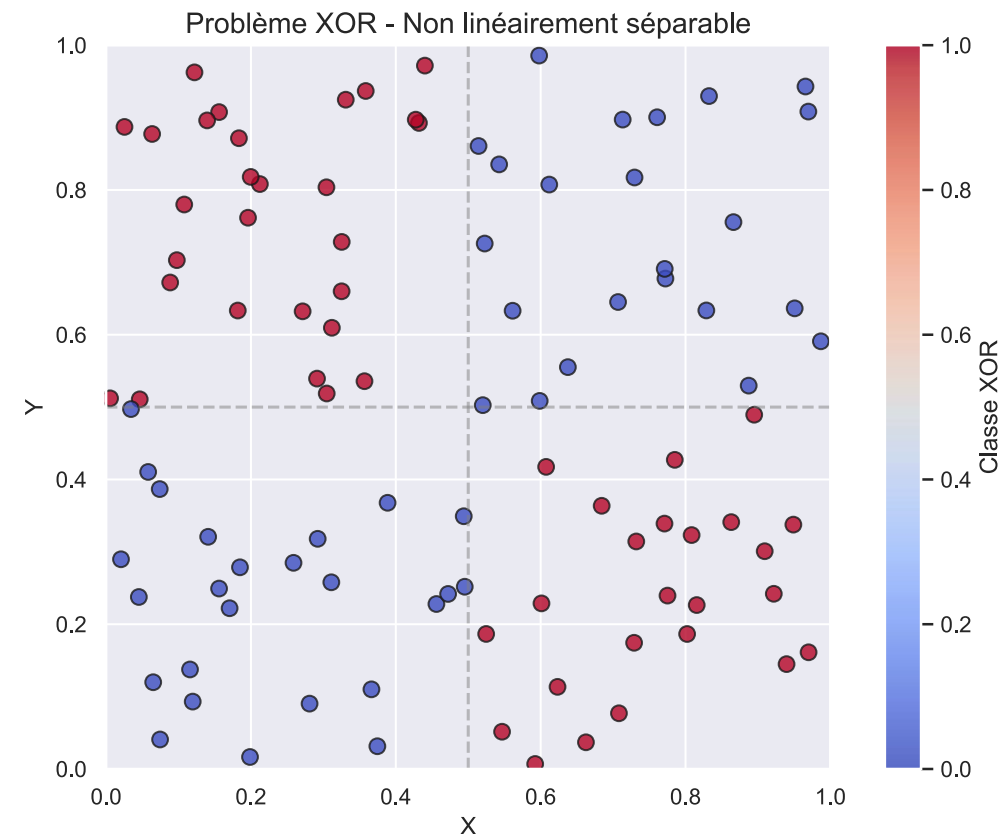


Réseau de perceptrons pour problèmes non linéaires

- Exemple de problème non linéairement séparable : le XOR (« ou exclusif »). Non solvable par un perceptron seul.
- Solvable par des perceptrons connectés entre eux : un réseau de neurones.



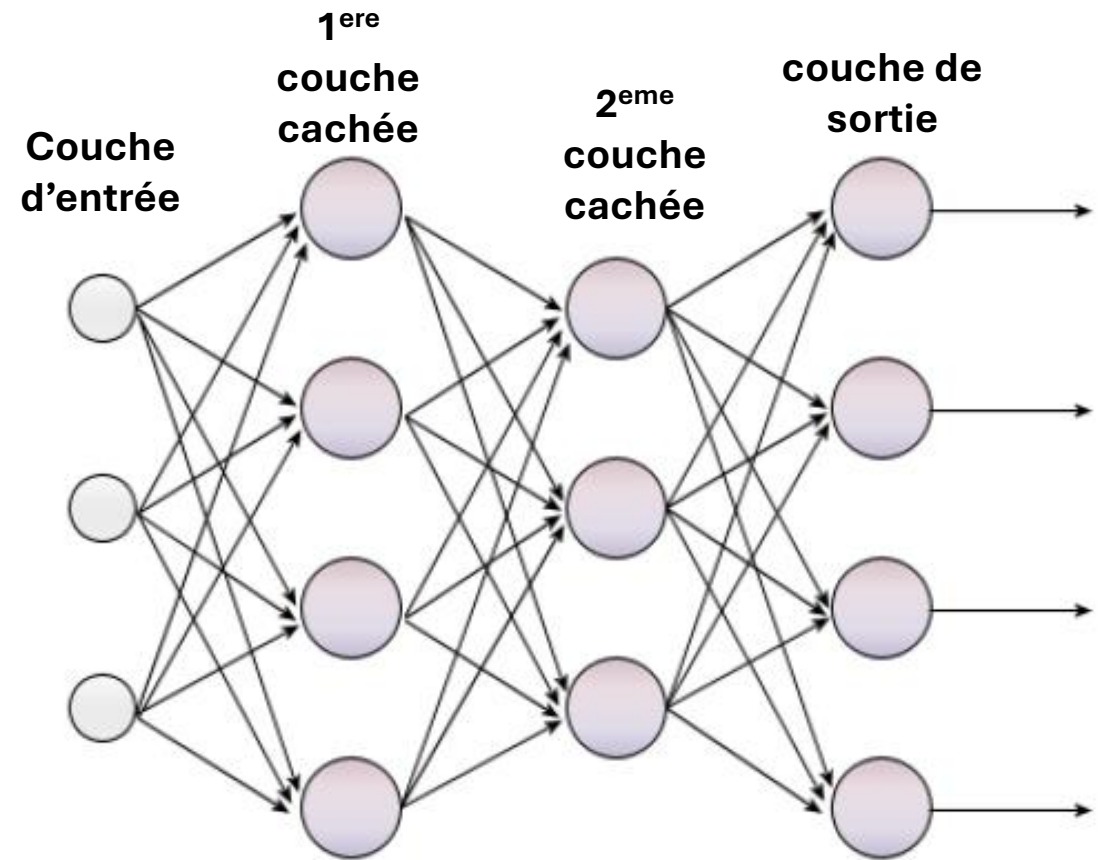
Peut être vu comme la transformation du SVM. Non-linéarité (cette fois-ci, apprenable) qui projettent l'entrée dans un espace où le problème est linéairement séparable.



Réseaux de neurones : le perceptron multicouche

Perceptron multicouche (MLP)

- Idée : connecter des perceptrons les uns à la suite des autres (en profondeur) afin de pouvoir résoudre des problèmes non linéaires.
- Les perceptrons sont organisés en couches, appelées couches *denses*.
- Les perceptrons de la couche $(k+1)$ reçoivent comme entrée les sorties des perceptrons de la couche k .

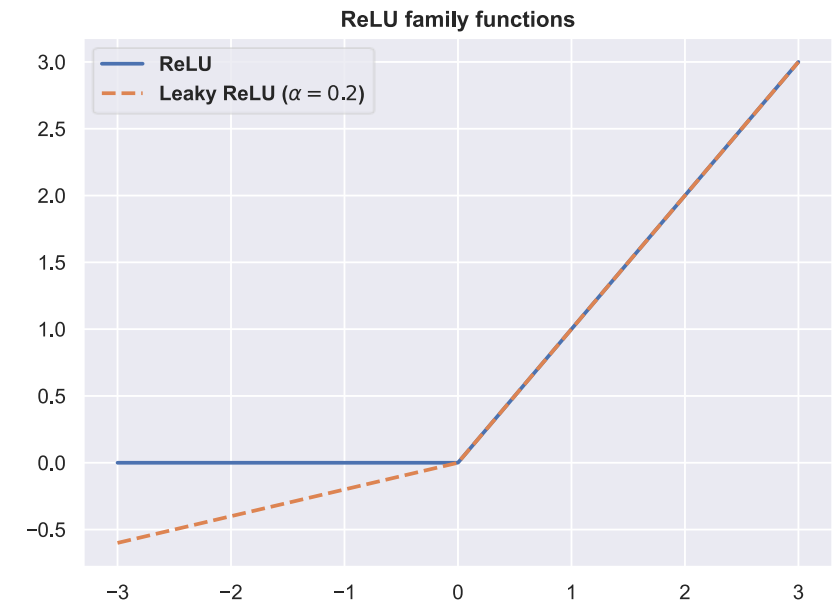
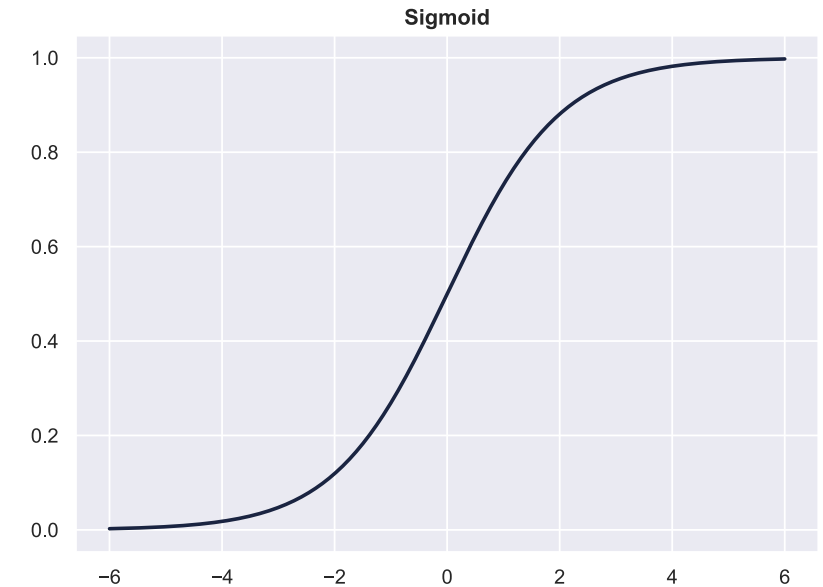
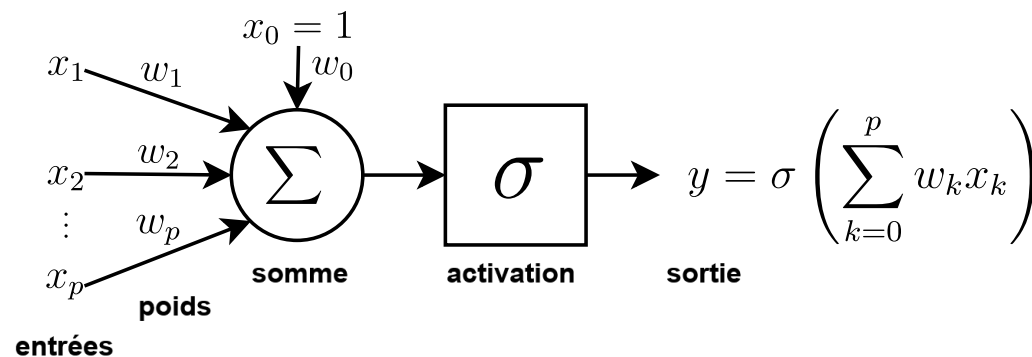


Perceptron multicouche à 3 entrées et 4 sorties, et 2 couches cachées

Fonction d'activation

On utilise d'autres fonctions d'activation (σ) en sortie du perceptron que la fonction de Heaviside. Par exemple :

- Sigmoide (fonction logistique) : $\sigma(x) = \frac{1}{1 + e^{-x}}$
- ReLU : $\text{ReLU}(x) = \max(0, x)$
- LeakyReLU : $\text{LeakyReLU}(x) = \max(\alpha x, x)$



Représentation matricielle d'une couche d'un MLP

- Soit une couche *dense* contenant n neurones et ayant p entrées et une fonction d'activation σ .
- Chaque neurone i calcule sa sortie y_i avec ses poids $(w_{i,0}, \dots, w_{i,p})$:

$$y_i = \sigma \left(w_{i,0} + \sum_{j=1}^p w_{i,j} x_j \right)$$

- En posant $\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_p \end{bmatrix}$, $\mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$, $\mathbf{W} = \begin{bmatrix} w_{1,1} & \cdots & w_{1,p} \\ \vdots & \ddots & \vdots \\ w_{n,1} & \cdots & w_{n,p} \end{bmatrix}$ et $\mathbf{b} = \begin{bmatrix} w_{1,0} \\ \vdots \\ w_{n,0} \end{bmatrix}$
(poids, weights) (biais)

- On peut réécrire la sortie de la couche comme:

$$\mathbf{y} = \sigma (\underbrace{\mathbf{W}\mathbf{x} + \mathbf{b}}_{\text{Linéarité (fonction affine)}})$$

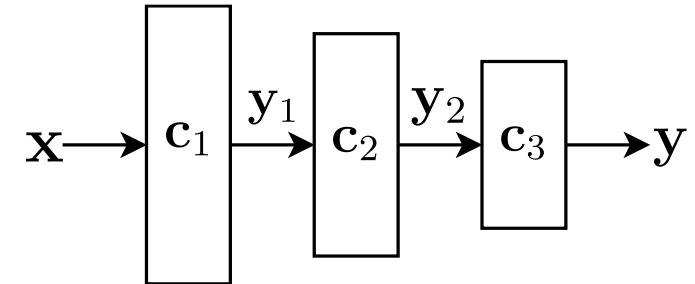
non-linéarité 

Représentation matricielle d'un MLP

- Un perceptron multicouche (MLP) peut alors être décrit comme une succession de c couches, où chaque couche k a sa propre matrice de poids $\mathbf{W}_k$, vecteur de biais $\mathbf{b}_k$, et fonction d'activation σ_k .

- Le calcul de la sortie $\mathbf{y}$ du MLP pour une entrée $\mathbf{x}$ est donnée par :

$$\left\{ \begin{array}{l} \mathbf{y}_0 \leftarrow \mathbf{x} \\ \forall k \in \llbracket 1; c \rrbracket, \quad \mathbf{y}_k \leftarrow \sigma_k(\mathbf{W}_k \mathbf{y}_{k-1} + \mathbf{b}_k) \\ \mathbf{y} \leftarrow \mathbf{y}_c \end{array} \right.$$



- Les différentes non-linéarités σ_k sont essentielles, car sans elles une composition de fonctions affines resterait une fonction affine.

Apprentissage ?

- Pourvu que l'architecture soit adaptée (hyperparamètres : nombres de neurones, nombres de couches, nombre de neurones par couche, fonctions d'activations), les perceptrons peuvent approximer des fonctions arbitrairement complexes.
- Encore faut-il trouver l'ensemble des poids et des biais de chaque couche qui permettent d'approximer cette fonction.
- Quel algorithme d'apprentissage pour ces modèles ?

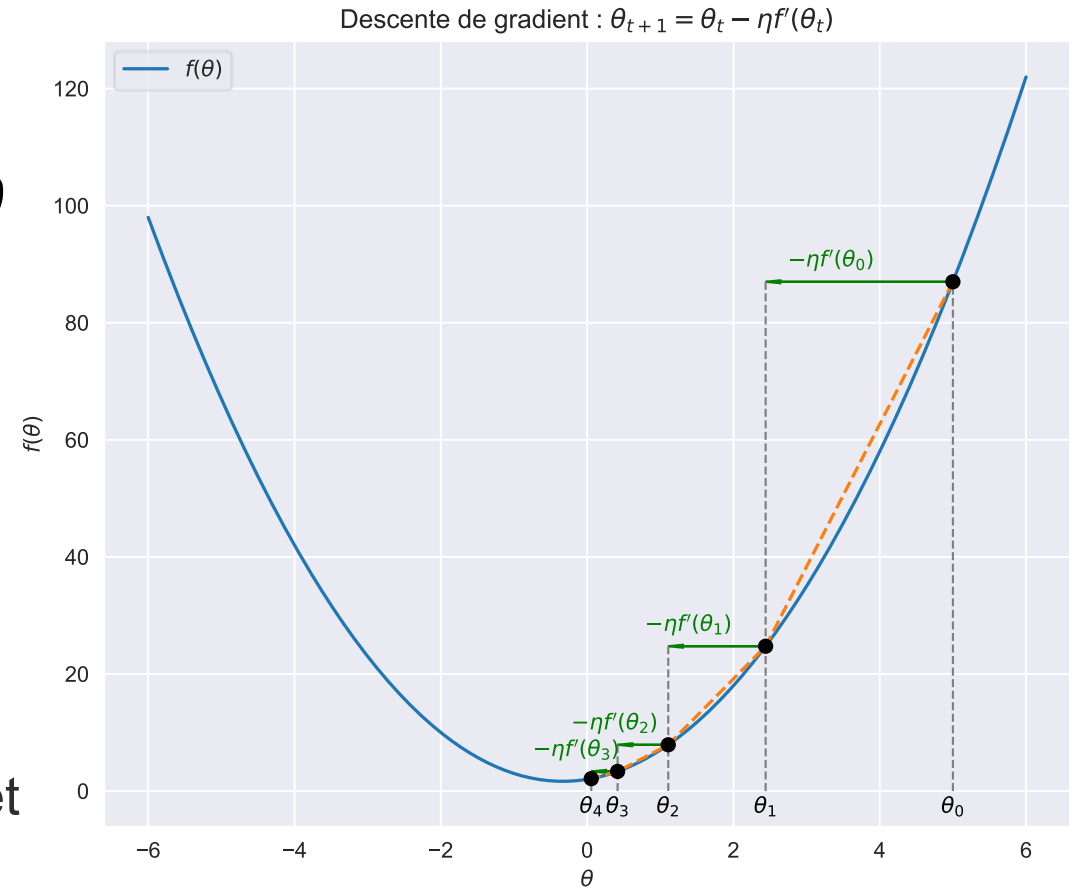
Apprentissage d'un MLP

Apprentissage supervisé d'un MLP

- On dispose d'un perceptron multicouche $\mathcal{M}_{\theta}$, où θ est un vecteur représentant l'ensemble des poids et biais contenus dans les couches du modèle. Le MLP possède p entrées et n sorties.
- On dispose d'un jeu d'entraînement $\{(\mathbf{x}_1, \mathbf{y}_1), \dots, (\mathbf{x}_N, \mathbf{y}_N)\}$ avec $\mathbf{x}_k \in \mathbb{R}^p$ et $\mathbf{y}_k \in \mathbb{R}^n$
- On veut minimiser une certaine fonction de coût/perte/« loss », notée $\mathcal{L}$.
Exemple :
$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{N} \sum_{k=1}^N (\mathcal{M}_{\theta}(\mathbf{x}_k) - \mathbf{y}_k)^2$$
- Comment trouver un jeu de paramètres θ^* qui minimise la fonction de perte ?

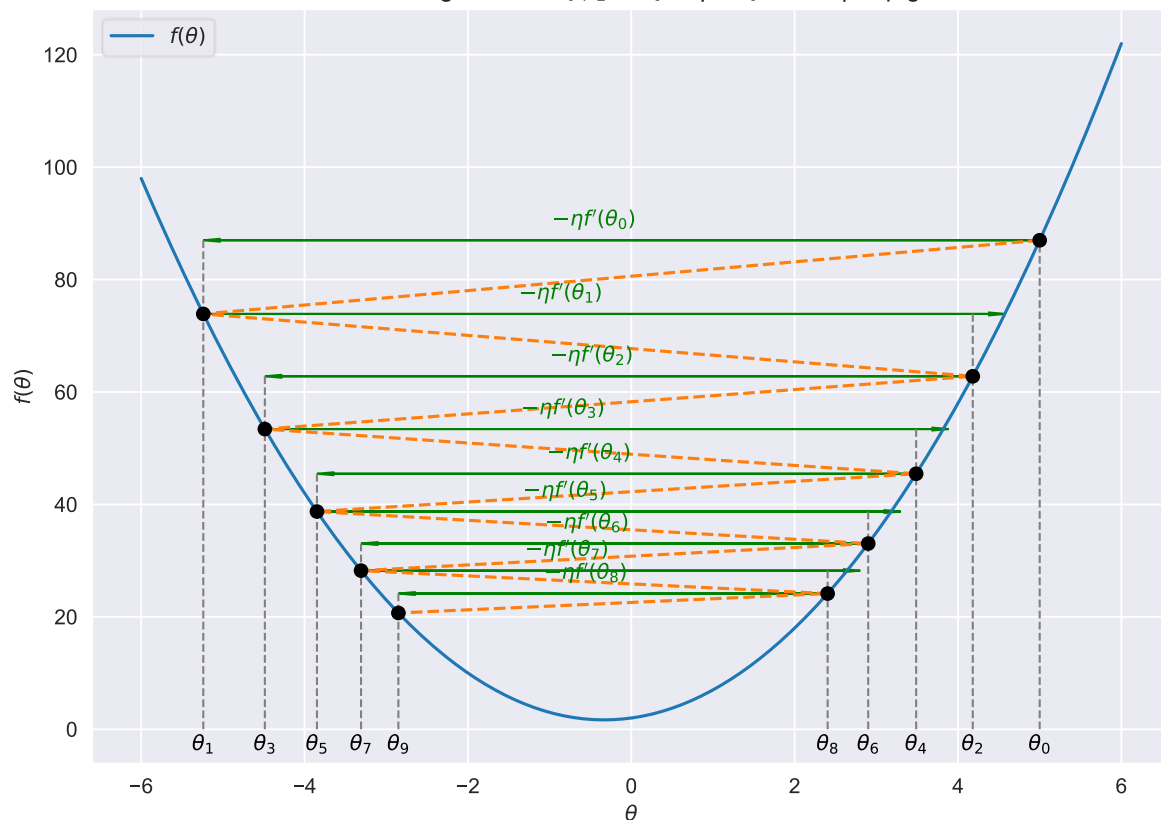
La descente de gradient

- Supposons qu'on cherche à trouver le minimum d'une fonction f dérivable et qui prend comme entrée un unique paramètre θ
- Idée : la dérivée f' nous indique dans quel sens f est croissante. Il suffit alors de se déplacer d'un petit pas dans la direction opposée.
- En prenant un petit pas d'apprentissage η et un état initial θ_0 , on effectue le processus itératif :
$$\theta_{t+1} = \theta_t - \eta f'(\theta_t)$$

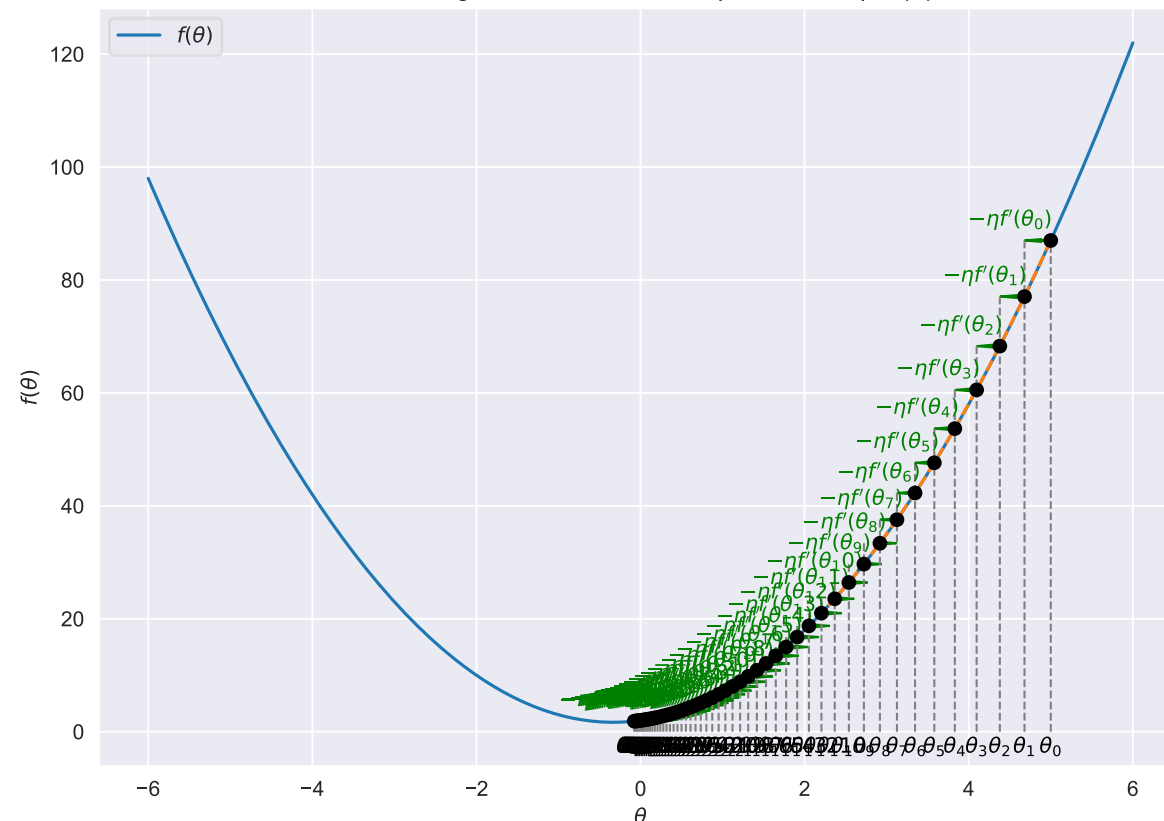


Descente de gradient : attention au choix du pas

Descente de gradient : $\theta_{t+1} = \theta_t - \eta f'(\theta_t)$ avec η trop grand



Descente de gradient : $\theta_{t+1} = \theta_t - \eta f'(\theta_t)$ avec η trop petit



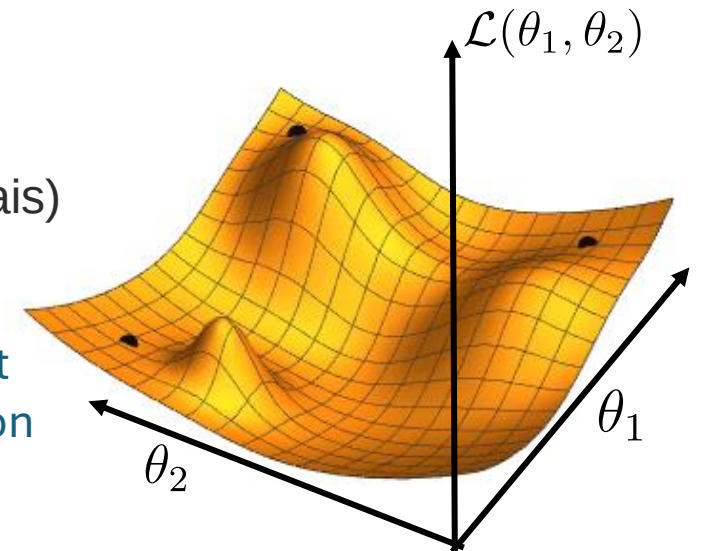
Descente de gradient de la fonction de perte

- Dans le cas de la fonction de perte $\mathcal{L}(\boldsymbol{\theta})$, on n'a pas une seule mais plusieurs entrées $\theta_1, \dots, \theta_d$. On réalise la descente de gradient sur chaque paramètre θ_i individuellement en utilisant la dérivée partielle $\frac{\partial \mathcal{L}}{\partial \theta_i}(\boldsymbol{\theta})$ (c'est la dérivée de la fonction qui fixe tous les paramètres sauf θ_i)

- La procédure itérative d'apprentissage est donc la suivante :
 - $\boldsymbol{\theta}_0$ est tiré aléatoirement (initialisation aléatoire des poids et des biais)

- $$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial \theta_1}(\boldsymbol{\theta}_t) \\ \vdots \\ \frac{\partial \mathcal{L}}{\partial \theta_d}(\boldsymbol{\theta}_t) \end{bmatrix}$$

← Le vecteur des dérivées partielles est appelé **gradient** et est souvent noté $\nabla \mathcal{L}(\boldsymbol{\theta}_t)$, il pointe en direction de la plus grande variation de la fonction. On « descend la pente ».



Calcul et rétropropagation du gradient

- Soit θ_i un des poids ou biais d'un des neurones du perceptron.
- Comment calculer $\frac{\partial \mathcal{L}}{\partial \theta_i}(\boldsymbol{\theta})$?
→ Algorithme de **rétropropagation du gradient** (*backpropagation*).
- Idée approximative : si une fonction compliquée (ex. h) est la composition de fonctions simples qu'on sait dériver (ex. f et g), alors on sait la dériver.

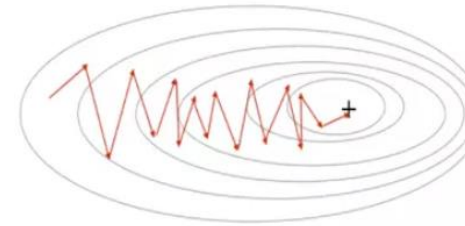
$$h(x) = f(g(x)) \quad \implies \quad h'(x) = g'(x)f'(g(x))$$

- Un MLP est une composition de couches simples qu'on peut dériver. Le gradient de l'erreur par rapport aux paramètres d'une couche est ainsi calculé grâce aux gradients des couches précédentes.

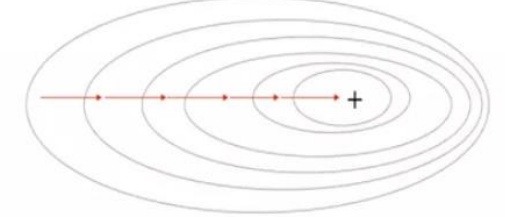
Gradient stochastique (SGD), batch et epoch

- Problème de la descente de gradient : calcul d'une fonction de perte sur tout l'ensemble d'apprentissage → extrêmement coûteux en calcul.
- Résolu par la descente de gradient stochastique (SGD). L'ensemble d'apprentissage séparés en sous ensembles appelés **batches** (par exemple, de taille 32). A chaque étape, on calcule la fonction de perte sur un des *batches*, puis on passe au suivant.
- Une **epoch** correspond au moment où tous les *batches* ont été utilisés. En général, l'ensemble d'apprentissage est alors mélangé, de nouveaux batches sont créés, et on commence une autre *epoch*.

Stochastic Gradient Descent



Gradient Descent



Optimiseurs : variantes de la SGD

- Pour éviter les minima locaux de la fonction de perte, ou plus couramment point-selle, des variantes de la SGD utilisent la mémoire des précédents gradients. Exemple : Momentum, RMSProp, Adam.

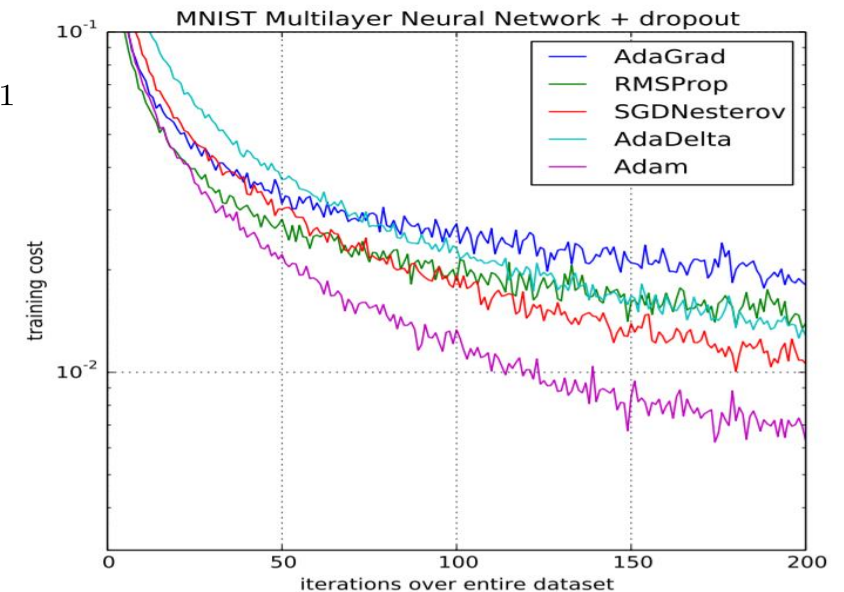
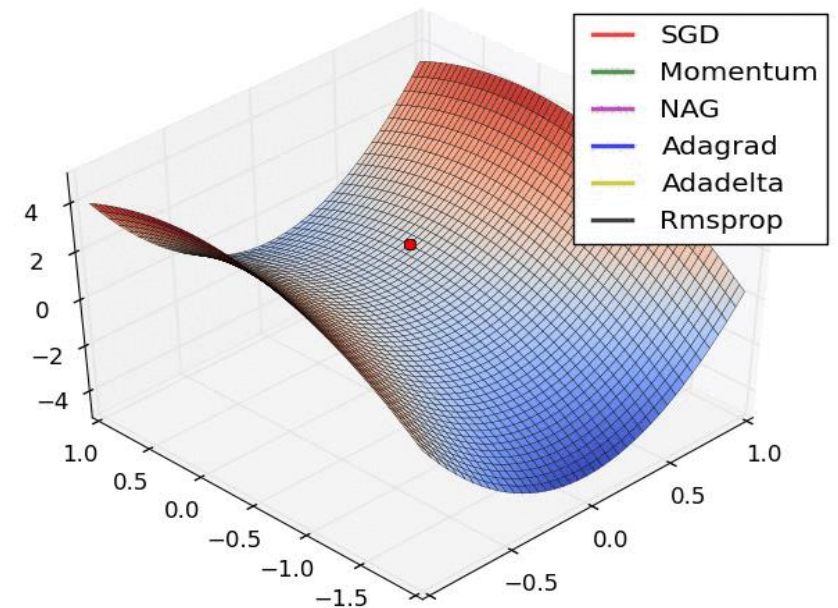
- **Momentum** (ajoute une inertie) :

$$\Delta\theta_{t+1} = \alpha\Delta\theta_t - \eta_t \nabla_{\theta} \mathcal{L}(\theta_t, \mathcal{B}_t) \quad ; \quad \theta_{t+1} = \theta_t + \Delta\theta_{t+1}$$

- **Adam** (le plus utilisé):

$$\mathbf{m}_{t+1} = \beta_1 \mathbf{m}_t + (1 - \beta_1) \nabla_{\theta} \mathcal{L}(\theta_t, \mathcal{B}_t) \quad ; \quad \hat{\mathbf{m}}_{t+1} = \frac{\mathbf{m}_{t+1}}{1 - \beta_1^{t+1}}$$
$$\mathbf{v}_{t+1} = \beta_2 \mathbf{v}_t + (1 - \beta_2) (\nabla_{\theta} \mathcal{L}(\theta_t, \mathcal{B}_t))^2 \quad ; \quad \hat{\mathbf{v}}_{t+1} = \frac{\mathbf{v}_{t+1}}{1 - \beta_2^{t+1}}$$

$$\theta_{t+1} = \theta_t - \eta_t \frac{\hat{\mathbf{m}}_{t+1}}{\sqrt{\hat{\mathbf{v}}_{t+1} + \epsilon}}$$



Utilisation d'un MLP

Perceptron multicouche pour la régression

- Le nombre de neurones de la dernière couche est la dimension du vecteur de sortie
- En général pas de fonction d'activation sur la dernière couche
- Fonction de perte :
 - **Mean Squared Error (MSE)** : La plus courante, convient bien aux distributions normales de données. Converge vers la moyenne des solutions.
 - **Mean Absolute Error (MAE)** : Moins sensible que la MSE en cas de valeurs aberrantes. Converge vers la médiane des solutions.
 - **Huber loss** : combine les deux.

Perceptron multicouche pour la classification

- Pour la classification binaire (2 classes):

- Un seul neurone de sortie et de fonction d'activation sigmoïde : $\sigma(x) = \frac{1}{1 + e^{-x}}$
- Fonction de perte : *binary cross-entropy* :

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

- Pour la classification n classes:

- **n neurones sur la dernière couche**, fonction d'activation *softmax* : $\text{softmax}(\mathbf{x}) = \frac{1}{\sum_{i=1}^n e^{x_i}} \begin{bmatrix} e^{x_1} \\ \vdots \\ e^{x_n} \end{bmatrix}$
- *Softmax* transforme un vecteur en un vecteur de valeurs entre 0 et 1 et dont la somme vaut 1 (peut être interprété comme une probabilité)

- Fonction de perte : *cross-entropy* :

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^n y_{i,j} \log \hat{y}_{i,j}$$

! Les labels doivent passer par un **one-hot encoding**, la classe j est représentée par un vecteur de zéros, sauf un 1 à la $j^{\text{ème}}$ position !